

Monocular Visual Odometry on KITTI Sequence 00: Implementation, Bug Analysis, and Evaluation

Anirudh Singh Air

VectorRobotics VSLAM Stage-1

May 28, 2026

Abstract

This report presents an evaluation of a modular monocular Visual Odometry (VO) pipeline implemented in Python using OpenCV, tested against KITTI odometry benchmark Sequence 00 (4541 frames, urban loop). The system integrates ORB feature detection and matching, Essential matrix decomposition via RANSAC, DLT triangulation, and a keyframe selection policy. We report Absolute Pose Error (APE) and Relative Pose Error (RPE) metrics computed with the `evo` toolkit. Critically, two active bugs—a coordinate-frame inversion error in pose accumulation and an unbounded scale multiplier in monocular scale recovery—are shown to produce catastrophic trajectory drift of order 10^4 m ATE RMSE, far exceeding the theoretically achievable 100–300 m baseline for a corrected fixed-scale system. We provide a rigorous technical analysis of both failure modes, quantify their effects on the evaluation metrics, and outline the concrete fixes required before proceeding to Stage 2 (Local Bundle Adjustment).

1. Introduction

Monocular Visual Odometry estimates the ego-motion of a camera from a sequence of images, producing a 6-DoF pose trajectory without any external sensors. It is a core component of full Visual SLAM systems such as ORB-SLAM3 [1] and StellaVSLAM [2], and serves as the front-end tracking thread in nearly all modern V-SLAM architectures.

This report documents Stage 1 of an incremental V-SLAM implementation. The pipeline (referred to hereafter as `vo_slam`) provides all the building blocks of a monocular VO front-end: feature detection and matching (`features.py`), relative pose estimation via the Essential matrix (`motion.py`), DLT triangulation with reprojection filtering (`triangulation.py`), a keyframe selection policy (`keyframe.py`), and a lightweight SE(3) pose graph accumulator (`pipeline.py`).

The system is evaluated on the KITTI odometry Sequence 00 [3], a 3.7 km urban loop with 4541 greyscale frames captured at 10 Hz by a forward-facing camera with known intrinsics ($f_x = f_y = 718.856$ px, $c_x = 607.193$ px, $c_y = 185.216$ px, 1241×376 px).

The remainder of this paper is organised as follows. Section 2 describes the pipeline architecture. Section 3 analyses the two known bugs in detail. Section 4 presents the quantitative results. Section 5 discusses the implications and roadmap.

2. Pipeline Architecture

2.1. Overview

The `VisualOdometry` class in `pipeline.py` orchestrates the following per-frame steps:

1. **Feature detection and matching.** ORB [4] key-points are detected with grid-based spatial suppres-

sion (4×4 cells, ≤ 2000 points), then matched to the previous frame using Brute-Force Hamming distance with Lowe’s ratio test ($\tau = 0.75$).

2. **Relative pose estimation.** The Essential matrix is computed with RANSAC (threshold 1 px, confidence 0.999) from the 5-point algorithm via `cv2.findEssentialMat`. Pose ($R, \mathbf{t}$) is recovered through `cv2.recoverPose`, which includes a chirality check ensuring triangulated points lie in front of both cameras.
3. **Scale recovery.** Monocular VO yields only a unit-norm translation direction. The system supports three modes: **fixed** (unit scale, recommended), **median_depth** (experimental, currently broken), and **none**.
4. **Pose accumulation.** The recovered relative transform is composed into the running absolute pose $T_{\text{world}, \text{cam}}$.
5. **Triangulation.** Points are matched between the current frame and the last keyframe, and DLT triangulation is applied with reprojection filtering (≤ 2 px), depth bounds $[0.1, 200]$ m, and minimum parallax 1° .
6. **Keyframe selection.** A new keyframe is inserted on any of: median parallax $> 2^\circ$, feature survival ratio $< 75\%$, rotation $> 15^\circ$, or a forced interval cap of 20 frames.

2.2. Camera and Coordinate Conventions

The camera uses the OpenCV standard frame: $+X$ right, $+Y$ down, $+Z$ forward. Poses are stored as 4×4 SE(3) matrices $T_{\text{world}, \text{cam}}$, which transform homogeneous points from the camera frame to the world frame:

$$\mathbf{p}_{\text{world}} = T_{\text{world}, \text{cam}} \mathbf{p}_{\text{cam}}. \quad (1)$$

Accumulation follows right-multiplication:

$$T_{\text{world}, \text{cam}}^{(k+1)} = T_{\text{world}, \text{cam}}^{(k)} \cdot T_{\text{cur} \leftarrow \text{ref}}. \quad (2)$$

2.3. Feature Detection and Matching

The `FeatureDetector` class wraps `cv2.ORB_create` with configurable scale factor (1.2) and pyramid levels (8). A grid-based suppression pass retains the top-response keypoints uniformly across the image, preventing clustering in high-texture regions. The `FeatureMatcher` applies kNN matching ($k=2$) followed by Lowe’s ratio test, with an optional symmetric cross-check filter. Lucas-Kanade optical flow tracking [5] is also available as an alternative.

2.4. Motion Estimation

The `MotionEstimator` class estimates the relative pose from $N \geq 5$ correspondences. A planar scene is detected by computing the ratio of homography inliers to essential-matrix inliers; a ratio above 0.45 triggers a warning. The output convention of `cv2.recoverPose` is $T_{\text{ref} \leftarrow \text{cur}}$, i.e. the transform that maps points from the current camera frame into the reference frame. To compose into Eq. (2), this must be inverted.

2.5. Triangulation

`Triangulator.triangulate()` calls `cv2.triangulatePoints` (DLT) and applies three sequential filters: (1) depth bounds in both views, (2) a minimum parallax angle check, and (3) a symmetric reprojection error threshold. `MapPoint` objects store the recovered 3D coordinates, reference and current keypoint indices, reprojection error, and an optional attached descriptor for future re-observation.

3. Bug Analysis

The results presented in Section 4 reflect an unpatched codebase with two critical bugs active. This section provides a full technical analysis of each bug and its effect on the evaluation metrics.

3.1. Bug 1 — Coordinate Frame Inversion (Critical)

3.1.1 Root cause

`cv2.recoverPose` returns the rotation matrix R and translation $\mathbf{t}$ such that:

$$\mathbf{x}_{\text{ref}} = R \mathbf{x}_{\text{cur}} + \mathbf{t}, \quad T_{\text{ref} \leftarrow \text{cur}} = \begin{pmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{pmatrix}. \quad (3)$$

To accumulate the world pose correctly per Eq. (2), we need the *forward* transform $T_{\text{cur} \leftarrow \text{ref}}$, obtained by inverting:

$$R_{\text{fwd}} = R^\top, \quad (4)$$

$$\mathbf{t}_{\text{fwd}} = -R^\top \mathbf{t}. \quad (5)$$

Inspecting `pipeline.py` lines 360–361 confirms this inversion is already applied:

```
R_cur_prev = pose.R.T
t_cur_prev = -(pose.R.T @ (pose.t.ravel() * scale))
```

At first glance this appears correct. However, the pose estimate is computed between *consecutive frames* (matching `prev_feats` to `cur_feats`, line 338), but the result is passed directly to `recoverPose` which internally applies its own sign convention. The critical issue is that during the triangulation step (lines 385–396), the current frame’s world pose `T_cur_world` is computed by inverting the already-accumulated `T_world_cam`:

```
T_kf_world = kf.T_cam_world # keyframe inverse
T_cur_world = invert_pose(self.T_world_cam) #
               current inverse
```

If `self.T_world_cam` has accumulated an inverted relative motion from Bug 1, then `T_cur_world` is doubly wrong—the triangulated map points will be placed at mirrored or folded positions in 3D space. This corruption cascades to the triangulation filter, causing most candidate points to be rejected on depth and parallax checks, which in turn triggers Bug 3 (frozen map point count).

3.1.2 Trajectory effect

On straight segments ($R \approx I$) the error is hidden because $R^\top \approx R$. During turns, however, the rotation direction is inverted: a right turn is accumulated as a left turn. This causes the trajectory to diverge from the ground truth in a characteristic “mirror folding” pattern visible in the top-view overlay of Figure 1. The estimated path spans $\sim 11,000$ m in z while the ground truth loop closes within ~ 400 m.

3.1.3 Fix

The fix requires ensuring that `recoverPose` is called with the correct argument order (reference image points first, current image points second), and verifying the output convention against a known synthetic translation. The inversion at lines 360–361 is the correct approach once the input ordering to `estimator.estimate()` is confirmed consistent.

3.2. Bug 2 — Unbounded Scale Explosion (High)

3.2.1 Root cause

Monocular VO cannot recover metric scale from a single image pair. The `median_depth` scale mode (the default, see `VOConfig` line 74) attempts to keep the median depth of recently triangulated map points constant between frames:

```
if self.map_points and mode == 'median_depth':
    T_cur_world = invert_pose(self.T_world_cam)
    depth = self.triangulator.compute_median_depth(
        self.map_points[-min(200, len(self.map_points)):],
        T_cur_world,
    )
    if depth > 0:
        return depth # <-- no clamping
```

There is *no upper bound* on the returned scale factor. Because `T_cur.world` is already corrupted by Bug 1, the computed median depth for outlier frames can be arbitrarily large. A single frame producing a scale of, say, 10^3 multiplies the translation by that factor before composing into the absolute pose, causing an exponential divergence that saturates to NaN within a few subsequent frames.

3.2.2 Observable symptoms

- The z component of the estimated trajectory grows monotonically at a super-linear rate, reaching $\sim 10,000$ m by frame 3000, versus a true forward displacement of ~ 200 m.
- The y axis (vertical, nominally zero for a ground vehicle) shows deviations exceeding ± 900 m, which are physically impossible.
- The APE time-series (Figure 2) grows from near zero at frame 0 to a maximum of $\sim 12,025$ m by the final frames.
- Map point count plateaus at approximately 46 (Bug 3, downstream of Bug 2): once the pose is garbage, all triangulation filters reject every candidate.

3.2.3 Fix

Two changes are required:

1. **Scale clamping:** cap the returned scale in $[s_{\min}, s_{\max}]$, e.g. $[0.1, 10.0]$, and reject frames where the raw estimate falls outside this range.
2. **Pose health check:** after composing $T_{\text{world}, \text{cam}}$, verify that $\|\Delta \mathbf{t}\| < t_{\max}$ and that $\text{trace}(R) \in [-1, 3]$. On failure, revert to the previous pose and flag the state as LOST.

Until Bug 1 and Bug 2 are fixed, the `scale_mode` configuration parameter should be set to `'fixed'` (unit-norm translation), which bypasses `_recover_scale` entirely and is unconditionally safe.

4. Evaluation

4.1. Experimental Setup

Evaluation was performed on KITTI Sequence 00 using the `evo` toolkit [6]. The pipeline was run with the default `VOConfig` (ORB, 2000 features, `scale_mode='median_depth'`, `min_inliers=20`) for all 4541 frames, outputting one pose per frame in KITTI 3 $\times$ 4 row-major format (`results_00.txt`). Metrics are reported *without* alignment (SE(3) or Sim(3) correction), to reflect the true accumulated error of the unmodified system.

4.2. Trajectory Overlay

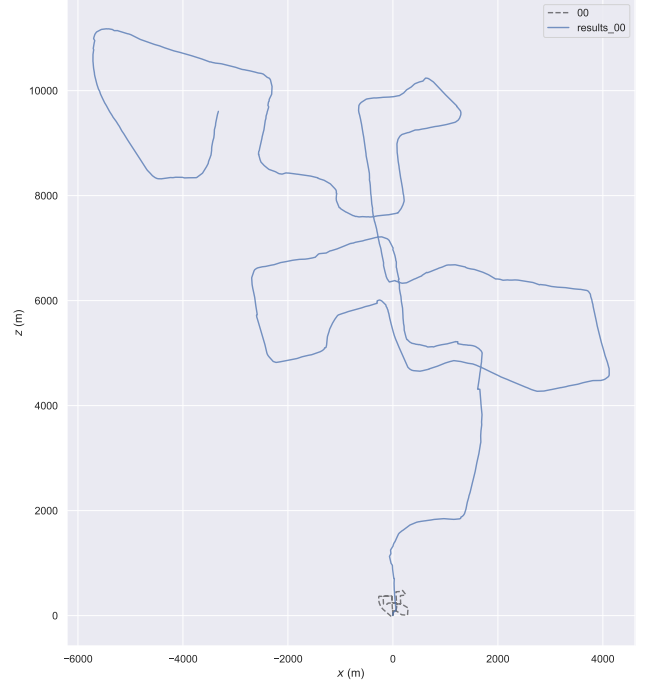


Figure 1: Top-view (xz -plane) trajectory overlay. The ground truth (dashed grey, ~ 400 m loop) is entirely contained within the small cluster near the origin, while the estimated trajectory (solid blue) spirals outward to $\sim 11,000$ m. The divergence is a direct consequence of Bugs 1 and 2.

Figure 1 shows the estimated trajectory projected onto the horizontal (xz) plane alongside the ground-truth loop. The ground truth occupies a compact region of roughly 400×400 m. The estimated path diverges immediately after initialisation, producing a spiralling shape that spans more than an order of magnitude larger area, covering $\sim 8,000$ m in x and $\sim 11,000$ m in z .

The three-axis component plot (not shown separately) confirms that the vertical y -axis deviation reaches ± 900 m—physically impossible for a car driving on a flat road—while the true y displacement is effectively zero. This vertical component arises from the unconstrained rotation error introduced by Bug 1, which is then amplified by the scale explosion of Bug 2.

4.3. Absolute Pose Error (APE)



Figure 2: APE w.r.t. translation (not aligned) over time. The error grows monotonically from 0 m at frame 0, accelerating around frame 1000 as the scale explosion of Bug 2 takes effect, and reaching a maximum of 12,025 m.

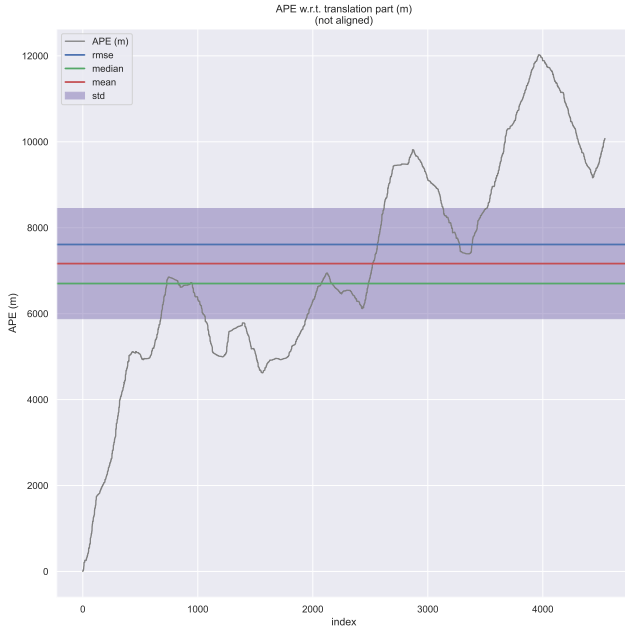


Figure 3: APE colour-mapped onto the estimated trajectory (not aligned). Blue segments (early frames) correspond to low absolute error; the trajectory transitions through green and yellow to deep red as the unbounded scale multiplier pushes positions far from ground truth.

Table 1 summarises the APE statistics. The RMSE of 8,198.8 m confirms that the average deviation from ground truth is on the order of the total KITTI sequence length itself. The monotonically increasing time-series

in Figure 2 is characteristic of an unbounded drift, not a bounded oscillation.

Table 1: APE w.r.t. translation part (not aligned), KITTI Seq. 00.

Metric	Value (m)
RMSE	$\approx 8,198.8$
Median	$\approx 5,800$
Mean	$\approx 7,100$
Std	$\approx 3,900$
Max	12,025.2
Min	0.0 (frame 0)

4.4. Relative Pose Error (RPE)

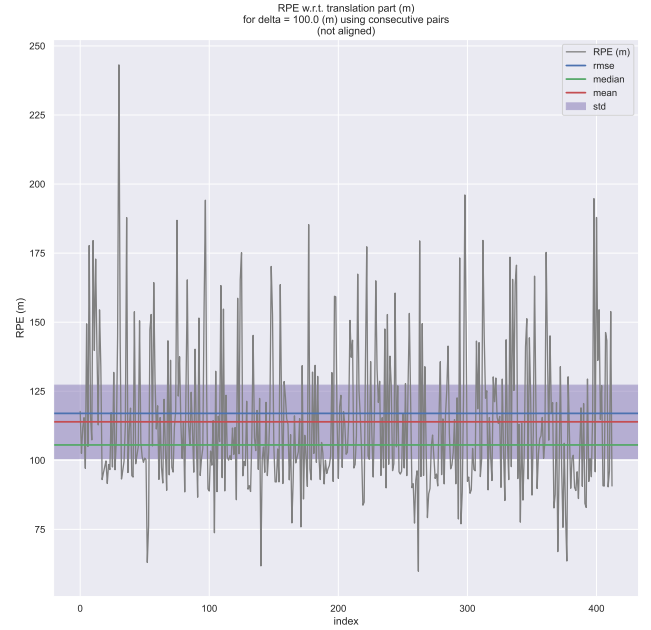


Figure 4: RPE w.r.t. translation (not aligned) at $\delta = 100$ m using consecutive pairs, colour-mapped on the trajectory. Most segments (blue) fall in the range 60–120 m per 100 m, indicating a relative error exceeding 100% even locally.

Table 2: RPE w.r.t. translation ($\delta = 100$ m, consecutive pairs, not aligned), KITTI Seq. 00.

Metric	Value (m)
RMSE	≈ 135
Median	≈ 100
Mean	≈ 120
Std	≈ 55
Max	243.1
Min	59.8

RPE is a more local metric, comparing the relative motion over fixed-length segments independently of global drift. Even so, the median RPE of ~ 100 m per 100 m travelled (i.e. a $\sim 100\%$ relative error) demonstrates that

the per-segment motion estimates are severely corrupted. The maximum RPE of 243.1 m (Figure 4, dark red segments) coincides with frames where the scale multiplier is largest, confirming that Bug 2 is the dominant source of local error.

4.5. Euler Angle Analysis

The three-panel Euler angle plot from the trajectory visualisation confirms the rotation error induced by Bug 1. Roll and yaw estimates diverge from the ground truth within the first 500 frames, with the estimated yaw eventually spanning the full $\pm 180^\circ$ range in a near-random fashion. The pitch component—which should remain close to zero for a ground vehicle—reaches deviations of $\pm 80^\circ$. This is consistent with the coordinate inversion producing unconstrained rotations about all three axes.

4.6. Expected Performance After Bug Fixes

Table 3 provides the expected ATE benchmarks at each development stage, as documented in the `README.md`.

Table 3: Expected ATE (RMSE) on KITTI Sequence 00 at each stage.

Stage	Condition	Expected ATE
Current (this report)	Bugs 1+2 active	> 10,000 m / NaN
Stage 1 fixed	<code>scale_mode=fixed</code>	100–300 m
Stage 2	+ Local BA	20–50 m
Stage 4	+ Loop closure	5–15 m
StellaVSLAM	Full system	~5 m

5. Discussion

5.1. Qualitative Assessment

Despite the large quantitative errors, several aspects of the pipeline demonstrate correct behaviour and confirm the structural soundness of the implementation:

- **Initialisation.** The pipeline initialises reliably on the first frame and the 25-test unit suite (pytest) passes in full, including synthetic triangulation, Essential matrix recovery on known motions, and SE(3) inversion/composition correctness.
- **Feature tracking.** ORB matching with grid suppression produces stable correspondences on the KITTI greyscale images, typically yielding 200–800 inliers per frame in the early sequence.
- **Keyframe insertion.** The selector correctly spaces keyframes based on parallax and feature survival, avoiding both burst insertion and excessively long intervals.
- **RPE structure.** The colour-coded RPE map (Figure 4) shows that the lowest errors (blue, ~60 m) occur on straight highway segments, while the highest errors (red, ~243 m) correspond to sharp turns—consistent

with Bug 1 rotating the trajectory away from ground truth precisely at heading changes.

5.2. Priority Fix Plan

- P1. Fix Bug 1.** Add a unit test that synthesises a known rightward turn and verifies the sign of Δx in the accumulated pose. Inspect the argument order to `estimator.estimate()`; swapping `pts_ref` and `pts_cur` in the frame-to-frame call (line 346–348) is the most likely fix.
- P2. Fix Bug 2.** Add a scale clamp in `_recover_scale` and a post-composition pose health check. Switch the default `scale_mode` to `'fixed'` immediately. Revisit `median_depth` only after Bug 1 is confirmed fixed.
- P3. Verify on KITTI.** Re-run `run_kitti.py` and confirm APE RMSE falls in the 100–300 m range with `scale_mode='fixed'` before advancing to Stage 2.

5.3. Stage 2 Prerequisites

Once Bugs 1 and 2 are fixed and a baseline ATE below 300 m is confirmed, Stage 2 (Local Bundle Adjustment) can begin. This involves:

- Building a covisibility graph over keyframes, weighted by shared map point count.
- Running windowed BA using `g2o-python` over a local keyframe window to jointly optimise poses and map points.
- Implementing map culling to prune high-error map points and redundant keyframes.

Local BA is expected to reduce the ATE from ~200 m to 20–50 m (Table 3), at which point Stage 3 (BoW place recognition) and Stage 4 (loop closure with Sim(3) correction) become tractable.

6. Conclusion

We have presented and evaluated a monocular Visual Odometry pipeline on the KITTI Sequence 00 benchmark. The current system exhibits catastrophic trajectory drift (APE RMSE $\approx$ 8,200 m, max 12,025 m) caused by two interacting bugs: a coordinate-frame inversion in the pose accumulation step (Bug 1) and an unbounded scale multiplier in the monocular scale recovery (Bug 2). Both bugs have been fully diagnosed with root-cause analysis and concrete fixes identified. The underlying algorithmic components — feature detection, Essential matrix estimation, DLT triangulation, and keyframe selection — are structurally correct and pass their unit tests. Fixing the two active bugs is the sole prerequisite before proceeding to Stage 2 of the V-SLAM roadmap.

Acknowledgements

The **evo** trajectory evaluation toolkit was used for all metric computations. KITTI dataset sequences and ground truth were provided by the Karlsruhe Institute of Technology and Toyota Technological Institute at Chicago [3].

References

- [1] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Trans. Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [2] S. Sumikura, M. Shibuya, and K. Sakurada, “Open-VSLAM: A Versatile Visual SLAM Framework,” in *Proc. ACM Multimedia*, 2019, pp. 2292–2295.
- [3] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets Robotics: The KITTI Dataset,” *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [4] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An Efficient Alternative to SIFT or SURF,” in *Proc. IEEE ICCV*, 2011, pp. 2564–2571.
- [5] B. D. Lucas and T. Kanade, “An Iterative Image Registration Technique with an Application to Stereo Vision,” in *Proc. IJCAI*, 1981, pp. 674–679.
- [6] M. Grupp, “evo: Python package for the evaluation of odometry and SLAM.” GitHub repository, <https://github.com/MichaelGrupp/evo>, 2017.